

AUDIO_DSP

核心模块原理详解

—— 从零基础到能调用、优化、修改 ——

移植自 Electrosmith DaisySP, 面向 STM32H7 嵌入式平台

第一章 aud_phasor —— DDS 相位累加器

1.1 这是什么东西？

aud_phasor

是整个库里最简单的模块，也是最重要的模块。它的功能只有一句话：按固定速度从0数到1，到了1就回到0重新数。就像钟表的秒针——每秒走一格，走到60就归零。

在数字信号处理中，这叫做"相位累加器"。这个名字听起来吓人，其实就是"角度累加器"。正弦波的完整周期是 2π 弧度（360度），如果我用一个从0到1的数代表角度（0=0度，0.25=90度，0.5=180度，0.75=270度，1.0=360度），那这个递增的数就精确地描述了正弦波的相位。

一句话总结：aud_phasor = 一个自动前进的"角度指针"，从0走到1，周而复始。

1.2 用具体数字走一遍

假设采样率 $f_s=1\text{MHz}$ （每秒1百万个采样点），我想生成一个 50kHz 的信号。每个采样周期，相位应该前进多少？

50kHz 意味着每秒5万个完整周期。 1MHz 采样率意味着每秒1百万个采样点。所以每个采样点，相位前进 $50000/1000000 = 0.05$ 。也就是说：

采样点0: $\text{phase}=0.00 \rightarrow \sin(0.00*2\pi) = 0$
采样点1: $\text{phase}=0.05 \rightarrow \sin(0.05*2\pi) = \sin(18^\circ) = 0.309$
采样点2: $\text{phase}=0.10 \rightarrow \sin(0.10*2\pi) = \sin(36^\circ) = 0.588$
...
采样点19: $\text{phase}=0.95 \rightarrow \sin(342^\circ) = -0.309$
采样点20: $\text{phase}=1.00 \rightarrow \text{归零!} \rightarrow \sin(0^\circ) = 0$

这就是DDS的核心思想。频率到相位增量的转换公式是：

$$\text{phase_inc} = \text{freq} / \text{sample_rate}$$

phase_inc就是"每个采样点相位前进多少"。freq/sample_rate比值越大→每个点走得更远→频率越高。

1.3 代码逐行看

```
typedef struct {
    float freq;           // 当前频率 (Hz)
    float sample_rate;    // 采样率 (Hz)
    float inc;            // 每采样点相位增量 (弧度)
    float phs;            // 当前相位 (弧度, 0~2PI)
} Aud_Phase;
```

phs存储当前相位，单位是弧度（0到 2π ）。inc存储每次process前进的量。

```
void Aud_Phase_SetFreq(Aud_Phase *p, float freq) {
    p->inc = AUD_TWOPi * freq / p->sample_rate; // freq→弧度的转换
}

float Aud_Phase_Process(Aud_Phase *p) {
    float out = p->phs / AUD_TWOPi; // 弧度→0~1归一化输出
    p->phs += p->inc;                // 相位前进
    if (p->phs > AUD_TWOPi) p->phs -= AUD_TWOPi; // 溢出回绕
    if (p->phs < 0.0f) p->phs = 0.0f;          // 防御
    return out; // 0~1的锯齿波
}
```

调用Process时，先拿当前的归一化相位(phs/ 2π)，然后phs加上inc前进到下一个采样点。如果phs超过 2π （走完一个完整周期），就减去 2π 回绕到0附近。返回0~1的归一化值，直接乘 2π 就是弧度，或者直接送给其他接受0~1输

入的模块。

使用示例：生成50kHz正弦波：

```
Aud_Phase p;
Aud_Phase_Init(&p, 1000000.0f, 50000.0f, 0.0f); // 1MHz, 50kHz, 初相0
float phase = Aud_Phase_Process(&p); // 0~1锯齿波
float sine = sinf(phase * 6.2831853f); // 转为正弦波
```

1.4 为什么要有这个模块？直接用 $\sin(2\pi \cdot f \cdot n / f_s)$ 不行吗？

可以。但Phase把"相位推进"和"波形计算"分开了。好处是：

- 同一个相位可以同时驱动多个波形（sin+cos同时出，做IQ解调）
- 相位可以随时读取、修改（PM/FM调制）
- 多个Phase可以共享同一个采样率，各自独立改频

第二章 aud_osc —— 多波形振荡器

2.1 朴素波形和混叠问题

如果你在DAC上输出一个100kHz的锯齿波，采样率只有500kHz，会发生什么？

锯齿波在数学上含有无限多个谐波（基频100kHz的1倍、2倍、3倍...）。但是500kHz采样率只能表示250kHz以下的频率（奈奎斯特定理）。高于250kHz的谐波——300kHz、400kHz、500kHz——会被"折回"到低频：

- 300kHz → 折到200kHz ($=500-300$)
- 400kHz → 折到100kHz（直接干扰基频！）
- 500kHz → 折到0Hz（变成直流偏移！）

这种高频谐波折回低频的现象叫"混叠"（aliasing）。听起来是刺耳的金属噪音，看起来是波形上的毛刺。

2.2 polyBLEP 是怎么解决的？

混叠的根源是波形跳变。理想锯齿波在瞬间从+1跳到-1——这个跳变里包含了所有高频谐波。polyBLEP的思路是：在跳变处，用一小段平滑的多项式曲线替代瞬时跳变。

具体的修正公式（以锯齿波为例）：

- 从+1到-1的跳变处，加上一段多项式修正项
- 这段修正只在跳变点附近（phase_inc范围内）生效
- 修正项 = $0.5*t*t$ 或 $-0.5*(1-t)*(1-t)$ ，t是离跳变点的距离

效果：原来无限谐波的锯齿波，变成了谐波在奈奎斯特频率之前就被大幅衰减的"干净"锯齿波——无混叠！

aud_osc提供了8种波形：朴素版SIN/TRI/SAW/RAMP/SQUARE各一个，以及polyBLEP版TRI/SAW/SQUARE各一个。对2023H题来说，用AUD_WAVE_POLYBLEP_TRI输出三角波是正确答案。

2.3 关键API

```
Aud_Osc osc;
Aud_Osc_Init(&osc, 1000000.0f);           // 1MHz采样率
Aud_Osc_SetWaveform(&osc, AUD_WAVE_POLYBLEP_TRI); // 带限三角波
Aud_Osc_SetFreq(&osc, 50000.0f);          // 50kHz
Aud_Osc_SetAmp(&osc, 0.5f);               // 幅度 (0.5→1Vpp)
float clean_tri = Aud_Osc_Process(&osc);   // 获取下一个采样
```

PhaseAdd方法可以对相位做偏移——这等于频率调制(FM)。IsEOR/IsEOC查询周期边界，做触发信号很有用。

第三章 aud_svf —— 状态变量滤波器

3.1 滤波器能同时输出五种信号

你可能会想：滤波器不就是低通/高通/带通中的一种吗？SVF (State Variable Filter) 的特殊之处在于——它一次处理输入后，同时产生五个输出：

- Low (低通) —— 留下低频，砍掉高频
- High (高通) —— 留下高频，砍掉低频
- Band (带通) —— 只留下中间一段频率
- Notch (带阻) —— 砍掉中间一段频率
- Peak (峰化) —— 提升中间一段频率

这些输出不是"选一个"，而是"全部同时可用"。处理一次输入后，你可以读任何输出。

3.2 双采样是什么？为什么重要？

SVF的内部结构是一组差分方程。在采样率不够高（相对于截止频率）时，这些方程会变得不稳定——输出爆炸或振荡。

双采样的做法是：每来一个输入采样点，SVF内部跑两次迭代（pass1和pass2），输出取两次的平均值。这等效于把采样率翻倍，大幅提升高截止频率下的稳定性。对2023H题来说：1MHz采样率跑100kHz滤波器， f_s/f_c 只有10倍——单采样可能不稳定，双采样就稳了。

3.3 核心参数和用法

频率设置：freq参数是归一化频率（截止频率/采样率），不是Hz！内部用sin函数做了频率弯折修正，保证高频响应准确。

```
Aud_Svf svf;
Aud_Svf_Init(&svf, 1000000.0f);    // 1MHz采样率
Aud_Svf_SetFreq(&svf, 50000.0f);   // 截止50kHz
Aud_Svf_SetRes(&svf, 0.707f);      // Q=0.707 (巴特沃斯)

Aud_Svf_Process(&svf, input_sample); // 处理一个采样点
float bp = Aud_Svf_Band(&svf);      // 读带通输出
float lp = Aud_Svf_Low(&svf);       // 读低通输出 (同时可用!)
```

3.4 和普通双二阶滤波器(biquad)的区别

- SVF：一次Process得到5种输出；参数（freq/res）随时改不产生噪声；双采样保证高频稳定
- Biquad：每次Process只有一种输出；频繁改参数会产生"拉链噪声"；高Q值可能不稳定
- 对2023H题：SVF做PLL前级预滤波是首选——你需要BP输出，可能需要动态调频率，而且频率可能很高。

第四章 aud_dcblock —— 在线直流阻断

4.1 为什么需要在线去直流？

AD采样的信号经常有一个直流偏置（比如ADC输出整体抬高了0.05V）。如果用批处理方法（攒4096点求平均再减去），在LMS自适应这种"逐点处理"的场景根本行不通——你必须来一个点就处理一个点。

DC Block就是一个极其简单的在线去直流滤波器：

$$y[n] = x[n] - x[n-1] + 0.99 * y[n-1]$$

三个数：当前输入 $x[n]$ 、上一次输入 $x[n-1]$ 、上一次输出 $y[n-1]$ 。运算量：2次乘加。

4.2 原理：为什么 $x[n]-x[n-1]$ 能去直流？

直流分量从 $n-1$ 到 n 是不变的： $x[n]$ 和 $x[n-1]$ 都包含相同的DC。 $x[n]-x[n-1]$ 把DC消掉了。但剩下的信号是个"差分信号"——幅度不对。 $0.99*y[n-1]$ 这一项把上一拍的输出"回流"进来，相当于一个一阶高通滤波器。增益0.99意味着截止频率极低（约 $0.0016*fs$ ），只挡直流，不伤交流信号。

Init函数接收sample_rate参数但实际不用——gain固定为0.99。也就是说DC

Block的截止频率不跟随采样率变化。对于大多数应用（采样率几十kHz到几MHz），0.99对应的截止频率都足够低。

```
Aud_DcBlock dc;
Aud_DcBlock_Init(&dc, 1000000.0f); // sr参数保留但未使用
float clean = Aud_DcBlock_Process(&dc, raw_sample); // 去直流
```

第五章 aud_onepole —— 一阶滤波器

5.1 最轻量的滤波器

一阶滤波器是你能想到的最简单的滤波器——内部只有一个状态变量。它和DC Block的结构很像，但截止频率可调，且支持LP/HP两种模式。

核心差分方程：

$$lp = (g * in + state) / (1 + g)$$
$$state = g * (in - lp) + lp$$

其中 $g = \tan(\pi * freq_norm)$ ， $freq_norm = \text{截止频率} / \text{采样率}$ 。

为什么用tan？因为从模拟滤波器到数字滤波器的“双线性变换”中，频率会弯曲——高频被压缩。tan()把这个弯折修正回来，让数字滤波器的实际截止频率和设定值一致。

5.2 什么时候用一阶而不是SVF？

- 需要极轻量的运算（一阶每采样~5次乘加，SVF~20次）
- 只需要LP或HP，不需要陡峭过渡带
- 相位失真不重要
- 快速平滑信号、包络追踪等非音频场景

```
Aud_OnePole f;  
Aud_OnePole_Init(&f);  
Aud_OnePole_SetFrequency(&f, 0.01f); // 截止=0.01*fs (归一化)  
Aud_OnePole_SetMode(&f, AUD_ONEPOLE_LOW_PASS);  
float y = Aud_OnePole_Process(&f, x);
```

第六章 aud_delayline —— 分数延时线

6.1 延时线不是整数采样点？

假设采样率1MHz，你想延时5.3个采样点。整数延时只能取5或6——差了0.3个采样点=300ns。分数延时线用线性插值解决了这个问题：

- 先取整数部分的两个相邻采样点 $a = \text{buf}[n-5]$, $b = \text{buf}[n-4]$
- 用小数部分0.3在这两点之间线性插值： $a + 0.3*(b-a)$

这样一来，5.3个采样点的延时就是精确的5.3，不是近似的5或6。

6.2 Hermite插值——比线性更平滑

线性插值相当于“在两个点之间画直线”。但如果延时在变化（比如LFO调制延时做合唱效果），直线连接的折角会产生微小的频谱噪声。Hermite三次插值用4个点拟合一条光滑三次曲线，边角更平滑，噪声更低。代价是计算量翻倍。

6.3 使用方式

需要外部提供缓冲区——库不帮你分配内存。这是因为STM32上动态分配(malloc)不可靠。

```
float buf[2048];           // 2K采样=2ms @ 1MHz
Aud_DelayLine dl;
Aud_DelayLine_Init(&dl, buf, 2048); // 绑定缓冲区

Aud_DelayLine_SetDelay(&dl, 3.7f); // 3.7个采样点延时
Aud_DelayLine_Write(&dl, input);   // 写入当前输入
float y = Aud_DelayLine_Read(&dl);  // 读出延时后的信号
```

分数延时对2023H题的意义：发挥部分要求相位控制5度分辨率。100kHz信号在1MHz采样率下，1个采样点=36度。不用分数延时的话分辨率只有36度，用了可以到0.1度以下。

第七章 `aud_adsr` —— ADSR 包络发生器

7.1 包络是什么？

按下钢琴键→声音渐强→后渐弱→松开后慢慢消失。这个“音量随时间变化”的曲线叫包络。ADSR用4段来描述：

- Attack（起音）：音量从0升到最大
- Decay（衰减）：音量从最大降到Sustain水平
- Sustain（延音）：按住键保持的音量
- Release（释音）：松键后音量降到0

`aud_adsr`不是简单的线性斜坡——它用指数衰减，模拟真实乐器的自然衰减。

7.2 指数衰减的原理

每一采样点，包络值按固定比例向目标值靠近：

$$x[n+1] = x[n] + D0 * (target - x[n])$$

D0是衰减系数，由时间参数计算： $D0 = 1 - \exp(\log(1/e) /$

$(time * sample_rate))$ 。这个公式保证了经过time秒后，值恰好衰减到原来的1/e（约37%）。Attack阶段还有shape参数（0=线性，>0=对数曲线，更快到达峰值）。

7.3 gate触发机制

`aud_adsr`的触发方式很特别——用gate信号而不是trigger脉冲：

- gate从0变1（上升沿）→ 进入Attack
- gate保持1 → Attack→Decay→Sustain
- gate从1变0（下降沿）→ 进入Release
- gate保持0 → Release→IDLE

这和键盘的行为完全一致：按下键=gate高，按住=延音，松开=释音。

第八章 aud_comb —— 梳状滤波器

8.1 梳状滤波器是什么？

把信号延时、乘一个系数、再加回原信号——这就是梳状滤波器。它的频率响应不是平坦的，而是像梳子一样有一排间隔相等的"齿"（峰和谷）。齿之间的频率间隔=采样率/延时采样点数。

反馈式（feedback）：输出=读延时线；写延时线=输入+增益*读值。声音越来越密、金属质感越来越强——这是Karplus-Strong拨弦和混响的核心。

前馈式（feedforward）：输出=输入+增益*读延时线；写延时线=输入。产生固定的梳状频率响应，没有反馈循环。

8.2 2023H题里的应用

什么场景需要梳状滤波器？发挥部分要求分离三角波——三角波含丰富谐波。如果两个三角波的基频有倍数关系，梳状滤波器可以让某一频率的谐波恰好落在"齿尖"而被增强、让另一频率的落在"齿谷"被消除——辅助分离。不过2023H的主线方案是IQ解调/FFT，梳状滤波作为辅助手段。

第九章 aud_fm2 —— 2算子 FM 合成

9.1 FM是如何产生声音的？

FM（频率调制）和广播FM收音机原理一样——用一个信号（调制器）去改变另一个信号（载波）的频率。但这里的频率是音频频率，所以产生的是新的音色而非无线电波。

aud_fm2有两个算子：carrier（载波，产生听得见的音调）和modulator（调制器，通过改变载波的频率来改变音色）。数学上：

```
mod_signal = sin(mod_phase)
carrier_phase += carrier_inc + mod_signal * index
output = sin(carrier_phase)
```

index（调制指数）控制FM深度——index越大，音色越亮（谐波越多）。ratio（频率比）控制谐波排列——ratio=2产生奇次谐波为主，ratio=3产生更多非整数次谐波。

9.2 和直接调频率的区别

你可能会想：我直接每采样点改振荡器频率不就行了？关键区别：FM是“相位调制”——改变的是相位增量，而不是频率值本身。数学上，对相位做正弦调制等价于对频率做余弦调制（导数关系）。aud_fm2实现的是相位调制方式（index乘到调制输出再累加到载波相位），这和经典的Yamaha DX7 FM合成器一致。

第十章 aud_pluck —— Karplus-Strong 拨弦

10.1 一根弦的数学模型

Karplus-Strong算法可能是最简单的物理建模：

- 初始化：用随机噪声填充一段缓冲区（模拟拨弦的初始激励）
- 循环：读一个采样→用滤波器衰减→写回缓冲区的另一端
- 输出：读出的采样就是弦的声音

本质：噪声"波形"在缓冲区里不断循环，每循环一次被滤波器衰减一点——就像弦的振动在空气中逐渐衰减。延时线的长度决定了频率：延时越长=频率越低。

10.2 两种滤波模式

递归模式（PLUCK_MODE_RECURSIVE）： $\text{preval} = (*fp + \text{preval}) * \text{coeff}$ 。

这是经典的K-S算法——每个采样点和前一个做加权平均，高频自然衰减，模拟弦的能量损耗。

加权平均模式（PLUCK_MODE_WEIGHTED_AVERAGE）： $\text{preval} = *fp * \text{coeff} + \text{preval} * (1 - \text{coeff})$ 。拉伸了衰减时间，适合长的延音效果。

10.3 为什么需要外部提供缓冲区？

aud_pluck不内部分配缓冲区——你需要提供：

- 一个float数组作为缓冲区
- 数组的长度决定了最大延音时间
- 频率越高，需要的缓冲长度越短

这样设计是为了避免在STM32上使用malloc（嵌入式实时系统的大忌）。

```
float buf[1024];
Aud_Pluck pluck;
Aud_Pluck_Init(&pluck, fs, buf, 1024, AUD_PLUCK_MODE_RECURSIVE);
float trig = 1.0f; // 触发拨弦
float s = Aud_Pluck_Process(&pluck, &trig); // 第一声触发，后续trig自动变0
```

第十一章 `aud_allpass` —— 全通滤波器

11.1 不过滤任何频率的滤波器

全通滤波器很特殊——它对所有频率的"幅度"不做任何改变，只改变"相位"。不同频率被延迟的时间不同（低频延得多、高频延得少），但每个频率的能量保持不变。

差分方程：

```
write = input + gain * read_delayed
output = -gain * write + read_delayed
```

`gain`控制相位偏移的程度：`gain=0`→无效果(信号直通)，`gain`越接近±1→相位偏移越大。延时长度控制"哪个频率被延迟多少"。

11.2 合唱/移相器的核心积木

如果把几个全通滤波器串联，每个的`gain`不同，就产生了"不同频率有不同延迟"的效果——这就是移相器(`phaser`)。如果把全通滤波器的延时长度用`LF0`调制——这就是合唱(`chorus`)。`aud Phaser`和`aud chorus`模块就建立在这类积木之上。

第十二章 `aud_compressor` —— 动态压缩器

12.1 压缩器做什么？

压缩器自动调节音量——太大声就压低，太小声就抬高（补偿增益）。核心参数：

- `threshold`（阈值）：超过这个dB才压
- `ratio`（压缩比）：比如4:1，输入超4dB输出只多1dB
- `attack`（起音时间）：多快开始压
- `release`（释音时间）：多快停止压
- `makeup_gain`（补偿增益）：把压缩后的信号整体提上去

`aud_compressor`基于Faust的经典压缩器算法。核心是SLOPE包络跟随器——它追踪信号的RMS电平，`attack`快(exp系数大)、`release`慢(exp系数小)，模拟人耳对声音响度的感知。

12.2 Sidechain（旁链）

旁链的意思是：用另一个信号来控制压缩量，而不是用被压缩的信号自己。典型应用：DJ话筒——当人说话时自动压低背景音乐音量（背景音乐走in，话筒走key/旁链）。`aud_compressor`支持：`Process(in, key)`——用key的音量来决定压缩量，压缩in。

第十三章 aud_reverbsc —— FDN 立体声混响

13.1 混响是怎么模拟的？

真实房间的混响是声音从墙壁无数次反射形成的密集回音。数字混响用"反馈延迟网络"（FDN）来模拟：8条延迟线并联，每条有不同的延迟时间（从几毫秒到几十毫秒不等），彼此交叉反馈。信号在里面循环衰减，模拟声音在房间里的多次反射。

13.2 为什么有8条延迟线？

如果只用1条延迟线→金属感的单一回声（像梳状滤波器）。2~4条→回声密度增加。8条→回声足够密集，听起来像自然的房间混响而不是离散的回声。每条延迟线的长度和反馈系数都是从预先优化的参数表里取的——这些参数经过精心调整，让8条线的反射时间不会产生"可察觉的节奏感"。

13.3 内存警告

ReverbSc是库中最占内存的模块——aux[]数组约395KB。STM32H743的AXI

SRAM有512KB，放得下但很紧。如果你同时用其他大模块（如oscillatorbank 33KB、chorus

20KB），需要合理规划内存布局。Keil的scatter file可以把ReverbSc放到单独的AXI SRAM段。

第十四章 aud_looper —— 多模式循环录音

14.1 四种模式

NORMAL: 录音不断叠加到已有循环上（无限叠加）。

ONETIME_DUB: 只覆盖一次循环就自动退回播放。

REPLACE: 直接替换缓冲区内容。

FRIPPERTRONICS: 每次循环衰减0.707（类似磁带延迟，每次回放音质变差一点）。

初次触发→开始录音(REC_FIRST)。再次触发→结束录音并开始循环播放(PLAYING)。再次触发→进入覆盖录音(REC_DUB/REPLACE)。支持反向播放、半速播放、1200采样点的交叉淡变窗口（消除"咔哒"声）。

14.2 窗口函数做什么？

当循环的终点和起点相接时，如果不做处理，衔接点会有"啪"的一声（波形不连续）。Looper在衔接点附近用 $\sin(\pi/2 * \text{position})$ 做淡入淡出——第一段fade out，第二段fade in。1200采样点是经验值——足够长以消除可听噪声，足够短以不产生明显的"停顿感"。

附录：完整 API 速查表

核心工具

aud_dsp, aud_phasor, aud_dcblock, aud_delayline

振荡器

aud_osc, aud_fm2, aud_oscillatorbank, aud_formantosc, aud_variablesawosc, aud_variableshapeosc, aud_vosim, aud_zoscillator, aud_blosc, aud_grainlet, aud_harmonic_osc

滤波器

aud_svf, aud_onepole, aud_tone, aud_allpass, aud_comb, aud_moogladder, aud_mode, aud_nlfilt, aud_soap

动态

aud_limiter, aud_compressor, aud_crossfade, aud_balance

效果器

aud_chorus, aud_flanger, aud Phaser, aud_tremolo, aud_overdrive, aud_wavefolder, aud_autowah, aud_pitchshifter, aud_decimator, aud_sampleratereducer, aud_bitcrush, aud_fold, aud_reverb3d

噪声

aud_whitenoise, aud_clockednoise, aud_dust, aud_particle, aud_fractal_noise, aud_smooth_random, aud_samplehold, aud_maytrig

物理建模

aud_pluck, aud_karplusstring, aud_stringvoice, aud_resonator, aud_modalvoice, aud_drip, aud_polypluck

鼓

aud_analogbassdrum, aud_analogsnaredrum, aud_synthbassdrum, aud_synthsnaledrum, aud_hihat

包络/控制

aud_adenv, aud_adsr, aud_line, aud_port, aud_metro

采样/循环

aud_granularplayer, aud_looper

工具

aud_jitter, aud_pitchshifter